

Predicting Pokemon Card Prices from Card Attributes

[GitHub Repo Link](#) | [Video Link](#)

CS 210: Data Management for Data Science

1 Project Definition

1.1 Problem Statement

This project is about estimating the current market price of an existing raw Pokemon card based on structured card attributes. The target variable is the current TCGplayer-style market price in U.S. dollars, and each row in the dataset represents a card-finish combination such as Normal, Holofoil, Reverse Holofoil, 1st Edition, or Unlimited. The main question is: given a Pokemon card's attributes, can a model provide a reasonable appraisal when a recent sale or direct price comparison is not available?

This problem matters because Pokemon cards have become a large collectible market, and card prices are not always easy to understand from the printed card alone. Many cards have similar gameplay statistics, but very different prices. Buyers, sellers, and collectors often rely on direct listings or price guides, but those do not always explain why one card is worth much more than another. An attribute-based model can help identify which factors are most useful for valuation and where those factors fail.

1.2 Connection to Course Material

This project directly connects to CS 210 because it required collecting, cleaning, transforming, storing, querying, and analyzing data. The raw data came from public JSON endpoints, so the first step was to flatten semi-structured product metadata into tabular rows. Product records and price records had to be joined by `productId`, and each card finish had to be treated as a separate observation because the same card can have different prices depending on its finish type.

The project also used database concepts from the course. After cleaning and feature engineering, the dataset was stored in a local SQLite database. SQL queries were used to explore price ranges, average price by set, Pokemon type, release year, finish type, rarity, secret rare status, and the most expensive card by year. Pandas and NumPy were used for data frame manipulation, while scikit-learn was used for regression modeling and evaluation.

2 Novelty and Importance

2.1 Importance of the Project

The main challenge addressed by this project is that collectible card prices are not explained by one simple attribute. A high-HP card is not automatically expensive, and a low-HP card is not automatically cheap. The Pokemon card market has many low-price bulk cards and a smaller number of expensive chase cards, which makes it a useful case study for skewed data, feature engineering, and model evaluation.

The project is useful because it separates two questions: whether ordinary structured card attributes can estimate common card prices, and whether those same attributes can explain expensive

collector cards. The answer from the results is mixed. The models perform reasonably on very cheap cards, but they underpredict expensive cards. That finding is important because it shows that collector value is driven by factors beyond basic gameplay attributes.

2.2 Excitement and Relevance

Pokemon cards are a familiar and active collectible market, which makes the project easier to understand than a purely abstract dataset. The project combines a real hobby market with data management skills from class. It is interesting because the analysis can reveal whether the card information printed on or attached to a card, such as rarity, type, finish, age, and set, is enough to explain market price.

This topic is also relevant because many people who buy or sell Pokemon cards want a quick way to estimate value. Even if the final model is not perfect, the project can still show which attributes matter most and when a buyer or seller should be careful about relying only on structured data.

2.3 Review of Related Work

The project originally considered the Pokemon TCG API as a metadata source because its documentation provides REST endpoints for cards, sets, types, subtypes, supertypes, and rarities [3]. For the implemented dataset, TCGCSV was used because it provides public TCGplayer-backed product and price exports without requiring private API credentials. TCGCSV documents Pokemon as category 3, groups as set-like collections, products as records with nested `extendedData`, and prices as separate market price objects that need to be joined to products by `productId` [1]. TCGplayer's own pricing documentation also describes a group pricing endpoint that returns fields such as `productId`, `lowPrice`, `midPrice`, `highPrice`, `marketPrice`, and `subTypeName` [2].

Past and existing card-price tools often focus on looking up prices, tracking collections, or displaying market values. This project is different because the goal is not just to look up a known card price, but to test whether the card's structured attributes can explain or predict price. The project also uses a time-aware evaluation split, which makes the evaluation more realistic than a random train-test split for cards released across many years.

3 Progress and Contribution

3.1 Data Utilization

The dataset was created using TCGCSV's public Pokemon group, product, and price endpoints. The importer downloads all Pokemon groups, then fetches each group's products and prices. Product metadata and price data are joined using `productId`. Rows without a positive `marketPrice` are removed, and rows are kept only when the product has usable card attributes such as card number, HP, and card type.

The raw dataset contains 33,112 card-finish rows and 21 columns. After cleaning and feature engineering, the final analysis dataset contains 32,433 rows and 31 columns. The machine-learning-ready file contains 32,433 rows and 129 columns after categorical variables are one-hot encoded.

The cleaning process focuses on making the dataset usable for SQL exploration and modeling. Dates are parsed from source timestamps, non-Pokemon rows such as Trainer and Energy cards are excluded, missing retreat cost is treated as zero, and product URLs are removed from the cleaned modeling file. The derived features include release year, card age in days, card position

ratio, secret rare indicators, finish flags, rarity score, evolution stage score, HP per attack, and retreat cost per HP.

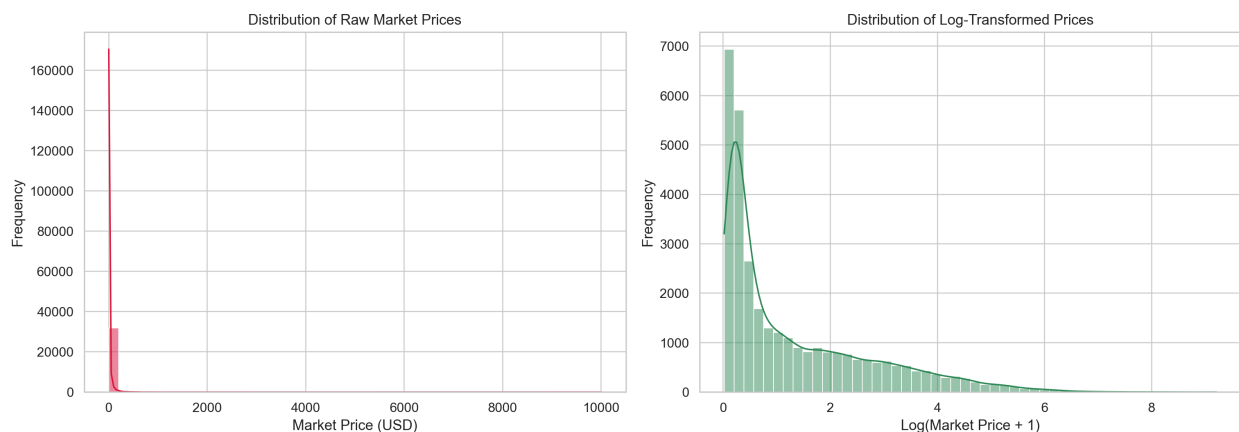


Figure 1: Raw and log-transformed market price distributions

The raw price distribution is heavily skewed. The median raw price is much lower than the mean, and the maximum observed price is \$10,000. The log-transformed distribution is more usable for modeling, which supports training the models on `log1p(price)`.

3.2 Models, Techniques, and Algorithms

The modeling task is regression because `market_price_usd` is continuous. Since the raw prices are highly skewed, the models are trained on `log1p(market_price_usd)` and predictions are transformed back into dollars for evaluation. This reduces the effect of extreme outliers during training while still allowing the final errors to be interpreted in dollars.

The project uses two baselines and two supervised models:

Model	Reason for Inclusion
Overall training median	Minimal baseline for a skewed price target
Rarity + finish median	Domain baseline using two strong collector-relevant categories
Ridge regression	Linear model with L2 regularization and an interpretable benchmark [5]
Random forest regressor	Tree ensemble that can model nonlinear relationships and feature interactions [4]

The project also uses SQL and visualization as analysis techniques. SQL queries compare price ranges, finish types, rarities, release years, and sets. Visualizations show the raw and log price distributions, price-band counts, and median price by finish type. These techniques help explain what the model is learning and where it fails.

3.3 Experimental Design

The main hypothesis was that card market price would be influenced more by collector-related attributes than by gameplay attributes. In other words, finish type, rarity, set age, set identity, and secret rare status were expected to matter more than HP, retreat cost, weakness count, resistance count, or attack count.

To evaluate this, the project uses a time-aware train-test split. The newest 10% of cards by age are held out for testing, while older cards are used for training. This is better than a purely random split because cards from the same set or era can be very similar. The notebook also uses `TimeSeriesSplit`, which scikit-learn recommends for time-ordered data where ordinary cross-validation could train on future observations and evaluate on older observations [6].

The evaluation metrics are mean absolute error (MAE), root mean squared error (RMSE), and R-squared. MAE is the easiest to interpret because it is measured in dollars. RMSE highlights large errors on expensive cards. R-squared measures explained variance, although it is sensitive to the extreme skew in card prices.

3.4 Key Findings and Results

The held-out test set is dominated by low-price cards. In the newest-card test split, 57.2% of rows are priced under \$1 and 70.5% are under \$5. Only 2.6% of held-out rows are above \$100. This imbalance matters because a model can look reasonable on cheap cards while still failing on the expensive collector cards that are most interesting for appraisal.

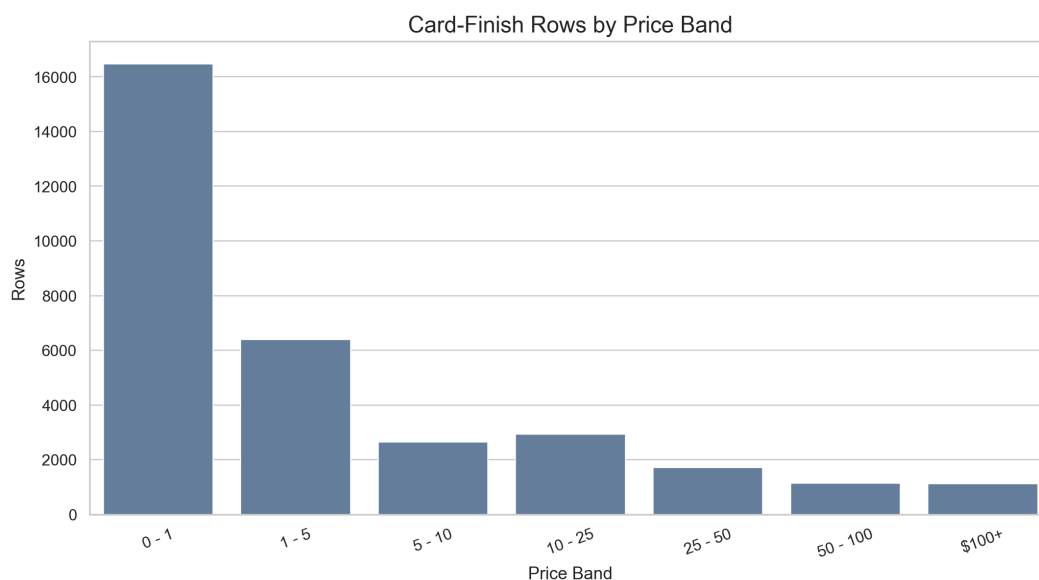


Figure 2: Card-finish rows by price band

The main held-out model results were:

Model	MAE	RMSE	R-squared
Baseline: overall train median	\$12.88	\$54.02	−0.0523
Baseline: rarity + finish median	\$11.65	\$51.85	0.0308
Ridge regression	\$11.73	\$51.85	0.0306
Random forest	\$11.74	\$51.27	0.0521

The most important result is that the machine learning models barely improve on the rarity-plus-finish median baseline. The random forest has the best RMSE and R-squared, but the

improvement is small. This suggests that the available structured attributes capture some market signal, especially finish, rarity, and age, but they do not fully explain Pokemon card value.

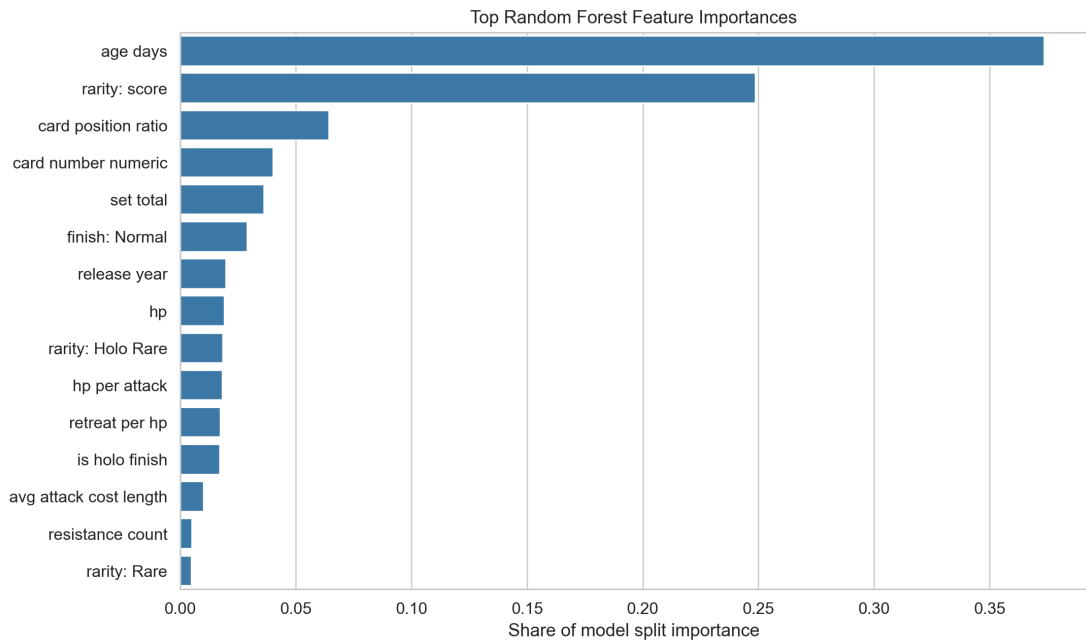


Figure 3: Top random forest feature importances

The feature-importance chart supports the project hypothesis. The random forest relies most heavily on age, finish type, rarity, set size, and card-position features. Gameplay attributes such as HP, retreat cost, weakness count, and attack count are less dominant, which suggests that collector-facing attributes carry more pricing signal than battle statistics.

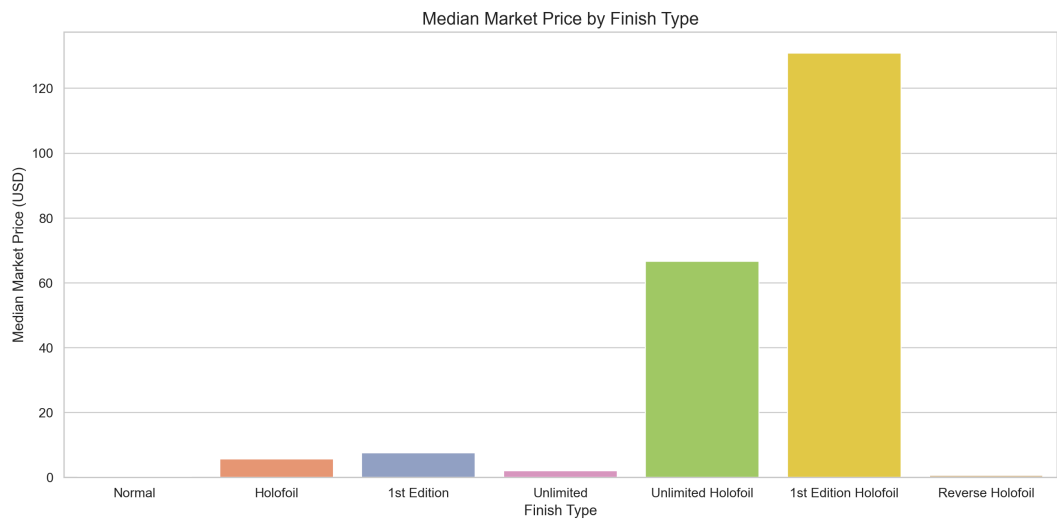


Figure 4: Median market price by finish type

The SQL and visualization sections show that finish type and rarity are more meaningful market features than gameplay statistics. Finish types such as 1st Edition Holofoil and Unlimited Holofoil have much higher median or average prices than ordinary Normal cards. Secret rare and special

illustration rare categories also stand out. By contrast, gameplay attributes like HP, retreat cost, weakness count, and attack count have weak direct relationships with price.

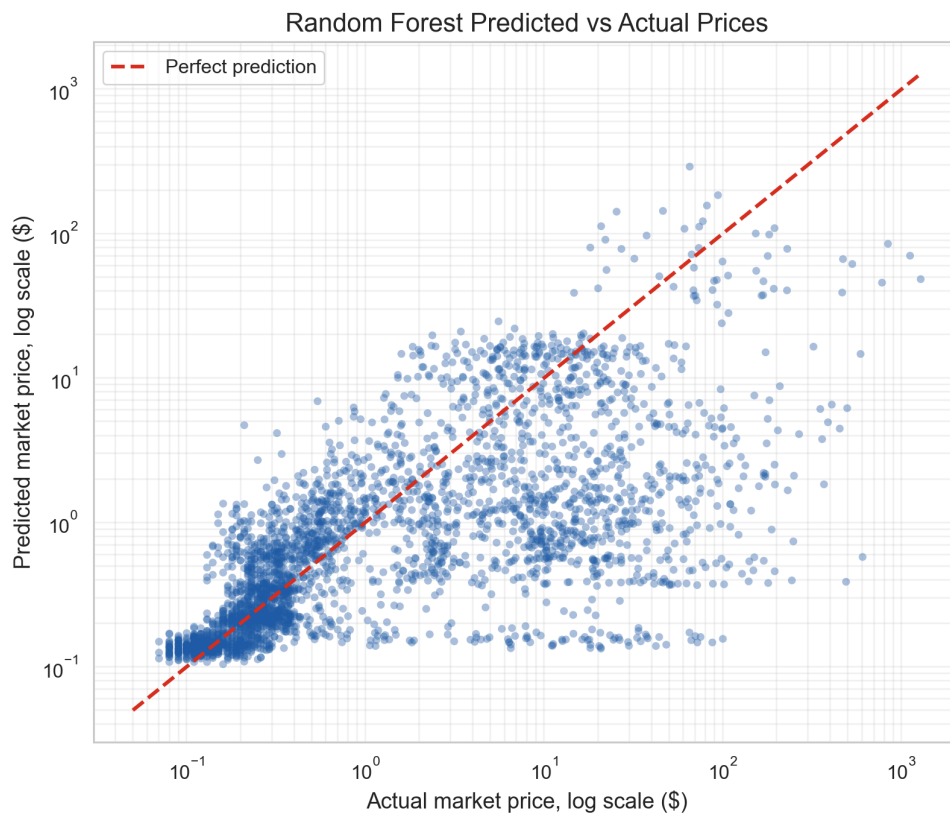


Figure 5: Random forest predicted versus actual market prices

The predicted-versus-actual chart shows why the overall metrics need careful interpretation. Most held-out cards cluster in the low-price region, where the model is often close enough for ordinary card appraisal. The expensive cards sit far below the perfect-prediction line, meaning the model systematically underpredicts high-value collector cards.

3.5 Evaluation

The Random Forest model has MAE of \$11.74, RMSE of \$51.27, and R-squared of 0.0521 on the held-out newest-card test set. Ridge regression has similar performance, with MAE of \$11.73, RMSE of \$51.85, and R-squared of 0.0306. The rarity-plus-finish median baseline performs almost as well as the machine learning models, with MAE of \$11.65 and RMSE of \$51.85.

The price-band analysis gives a clearer interpretation. For cards under \$1, the random forest has an error of only a few cents on average and most predictions are within \$1. For cards above \$100, the random forest has an average error above \$200 and predicts a median price of only a few dollars against an actual median above \$170. This shows that the model works much better for common low-price cards than for expensive chase cards.

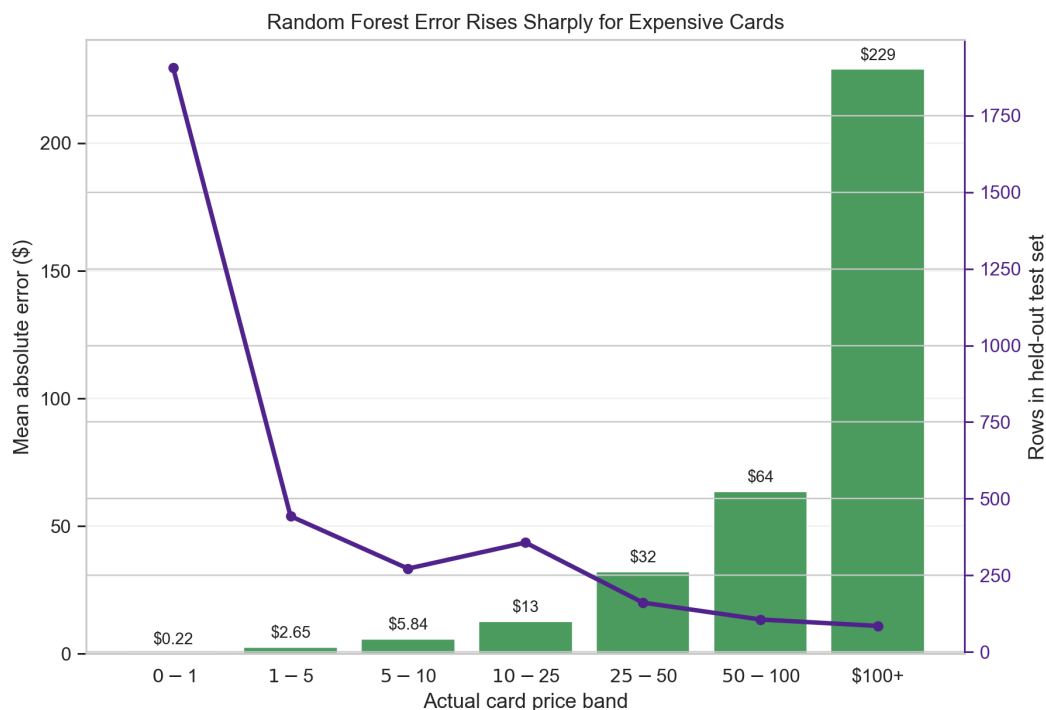


Figure 6: Random forest prediction error by actual price band

This price-band visualization makes the failure pattern more visible. The held-out test set contains many cheap cards and relatively few expensive cards, but the MAE grows from cents on sub-\$1 cards to more than \$200 for cards above \$100. This confirms that the model is useful as a baseline appraisal tool for common cards, but not reliable as a standalone estimate for rare chase cards.

The time-based cross-validation results also show instability across eras. The random forest has an average MAE of \$12.99 and average RMSE of \$50.87 across five time-based folds, but the average R-squared is negative because early validation folds contain expensive cards that the model struggles to predict. This reinforces the conclusion that the available attributes are not enough to fully explain high-end collector prices.

3.6 Advantages and Limitations

One advantage of this project is that it uses a real public dataset instead of a fully synthetic dataset. The data pipeline is reproducible, and the notebook creates cleaned CSV files, a machine-learning-ready CSV, and a SQLite database from the raw dataset. Another advantage is that the evaluation is honest about model limits. The project does not only report overall metrics; it also compares baselines, uses time-based validation, and analyzes errors by price band.

The biggest limitation is that the dataset does not include several factors that strongly affect collector value. The model does not know whether a Pokemon is especially popular, whether the artwork is iconic, whether a card has historical competitive importance, whether a card is graded, whether the raw card is damaged, or whether recent sales volume is high. Older cards may also have prices influenced by condition availability, while newer cards may reflect short-term release hype. TCGCSV's documentation also notes that market price can be null or slow-moving for products with very low sales volume [1].

Another limitation is that rarity labels are not perfectly consistent across Pokemon eras. Newer rarity names such as Illustration Rare and Special Illustration Rare do not map cleanly onto older categories. The project partly handles this with a manual rarity score, but a better production model would need a more historically aware rarity mapping.

4 Changes After Proposal

4.1 Differences from Proposal

The proposal originally mentioned using both the Pokemon TCG API and TCGCSV. During implementation, the project focused on TCGCSV because it provided both product metadata and market price data in a public format without private credentials. This made the dataset more practical and reproducible for a student project.

The proposal also described linear regression and a tree-based regression model. The final implementation uses Ridge regression instead of ordinary linear regression because Ridge is a regularized linear model and is more stable with many encoded categorical features. The tree-based model is Random Forest, as planned.

The final notebook also goes beyond the proposal by adding SQLite storage, multiple SQL exploration queries, feature engineering, baseline models, time-based cross-validation, and price-band error analysis. These additions make the evaluation stronger and make the results easier to interpret.

4.2 Bottlenecks and Challenges

The first major challenge was turning nested product metadata into a clean tabular dataset. TCGCSV product records store many card attributes in `extendedData`, which behaves like a list of key-value pairs. The importer had to flatten those fields, parse numbers like HP and card number, identify Pokemon card rows, and join each product to one or more price rows.

Another challenge was handling the skewed target variable. Most cards are cheap, but a few cards are extremely expensive. This made raw-price modeling difficult and required log-transforming the target, using medians as baselines, and evaluating performance by price band.

A final challenge was reproducibility. The notebook initially depended on cleaned files before creating them. This was fixed by reordering the notebook so it runs from raw CSV to cleaned CSV, then SQLite, SQL exploration, visualization, and modeling. The repository now includes `requirements.txt`, regenerated derived CSV files, and a Typst report source.

5 Conclusion and Future Work

5.1 Summary of Contributions

The project produced a complete data pipeline for Pokemon card price analysis. The importer collects data from TCGCSV and creates a raw card-price dataset. The notebook cleans and engineers features, stores the result in SQLite, runs SQL exploration, creates visualizations, trains baseline and machine learning models, and evaluates model errors using both holdout testing and time-based cross-validation.

The main analytical contribution is showing that structured card attributes can explain common low-price cards better than expensive collector cards. Finish type, rarity, age, and set-related information are useful, but they do not fully capture popularity, nostalgia, artwork, condition,

grading, or short-term collector demand. If this was completed as a group project, this section should be updated with each member's specific contributions before final submission.

5.2 Future Directions

Future work should add features for Pokemon popularity, card artwork, recent sales volume, condition, grade, and set-level collector demand. A stronger model could also separate cheap cards and expensive chase cards into different modeling tasks because the error patterns are very different across price ranges.

Another future direction is to create a small user-facing appraisal tool. A user could enter a card's set, rarity, finish, HP, type, and release year, and the tool could return a baseline estimate, model estimate, and warning when the card belongs to a price band where the model is unreliable.

6 References

- [1] TCGCSV. "TCGCSV Documentation." Accessed May 4, 2026. <https://tcgcsv.com/docs>
- [2] TCGplayer. "List Product Prices by Group." Accessed May 4, 2026. https://docs.tcgplayer.com/reference/pricing_getgroupprices
- [3] Pokemon TCG API. "Pokemon TCG API Documentation." Accessed May 4, 2026. <https://docs.pokemontcg.io/>
- [4] scikit-learn. "RandomForestRegressor." Accessed May 4, 2026. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
- [5] scikit-learn. "Ridge." Accessed May 4, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Ridge.html
- [6] scikit-learn. "TimeSeriesSplit." Accessed May 4, 2026. https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.TimeSeriesSplit.html